

Retrieval Refuses, a Small Model Chooses: Measuring a Local Chooser for Agents Bounded by a Declared Catalog

Rafael Uchôa

September 2026

Abstract

An agent whose only tools are the use cases a program declares cannot offer what the program does not do, provided something at its boundary enforces the declaration. We study the one step of such a design that needs a language model: choosing one declared use case for a person’s message, or choosing none. The chooser is a 270M-parameter function-calling model running on a laptop, placed behind a BM25 shortlist of four candidates and a score threshold, with every answer checked by code against the catalog and staged for the person rather than executed. Across ten experiments, each declared in writing before any code was written, we find that fine-tuning takes the model from 2 of 16 correct choices to between 72% and 90% on unseen, model-written tests, but that changing only the training seed moves that figure by 13 points and the model’s refusal rate by 52. No training set taught the model to refuse reliably; a threshold on the retrieval score, fitted on training data alone, refuses 88% to 94% of off-catalog messages across seeds and collapses the spread to 6 points. The retriever by itself names the right use case 85% of the time, which leaves the tuned model about one message in thirty-nine on this catalog. A use case the model never saw in training is chosen for 2 of 12 messages, so adding a use case is a build step, not a declaration. We report these numbers with their spreads, the signals that flag the model’s errors without a second model, and what does not generalise from a single catalog of thirteen use cases.

1 Introduction

A general-purpose language model given open tools does what the model does: it offers things the program cannot do and reaches for tools the program never declared. Tightening the prompt moves the problem without closing it, because what the prompt forbids the next message can ask for [8, 17]. A program that already declares its capabilities as contracts has a better boundary available: the contracts themselves.

We consider a program that declares what it can do as a *catalog* of use cases. Each use case is a command or a query with a contract: what it accepts, what it requires, what it leaves behind. Every way into the program, a command line, a screen, a tool protocol, is derived from the catalog. An agent is one more derived way in. It is handed the catalog as its tools and nothing else: no shell, no files, no free text to the person. A language model may sit in exactly two places. It may *choose* one use case and bind its arguments, or choose none; or it may run *inside* a use case, with its output held to a schema derived from the catalog. What keeps the agent inside the catalog is code at the boundary, never a prompt.

This paper measures the first of those two places. Choosing is the step the whole design rests on: if it needs a hosted model, every program derived from a catalog needs a key, a bill and a network at its boundary. The question is narrow:

Can a 270M-parameter function-calling model, running on a laptop, pick the right use case from a catalog of thirteen, and say nothing when the person asks for something the catalog does not do?

The second half is not decoration. A chooser that always chooses is an agent that invents capability, which is the failure the design exists to prevent. In the vocabulary of dialogue systems, the chooser is an intent classifier with out-of-scope detection [24], and the out-of-scope half is known to be the hard one [24, 43].

Contributions.

- A design in which the agent’s tools are derived from a declared catalog and every model answer is checked by comparison, not judged (§3).
- Ten experiments on one catalog, each declared before its code, nine of them with a fixed yes-or-no signal, measuring a tuned 270M chooser behind a retrieval shortlist (§5).
- The finding that refusal behaves as a retrieval problem: a threshold on the shortlist’s own score does what no training set did, and is the only stage whose number is reproducible across seeds (§5.2).
- A retrieval-only baseline that nearly matches the tuned model, and the observation that the quantity the model adds is the one that degrades as the catalog grows (§5.3, §5.6).
- Signals computed at no extra cost that flag the model’s errors, most of all when a use case is new to it (§5.5).

2 Related work

Tool use and function calling. Language models have been taught to call tools by self-supervision [35], by fine-tuning on large API collections with a retriever in front [28, 30], and evaluated on benchmarks of tool-augmented dialogue [1, 25]; Qu et al. [31] survey the area. Small on-device function-calling models are close to our setting: TinyAgent [11] and Octopus v2 [6] fine-tune models of a few billion parameters or fewer for a fixed tool set. Our chooser is a 270M model from the Gemma 3 family [14]. What we add is not a better tool caller but a measurement of which parts of choosing a small model is needed for, and which parts code and retrieval already do.

Retrieval before choosing. Offering every tool to a model scales poorly, both because long contexts are used unevenly [26] and because tool descriptions crowd the prompt [12]. Retrieving candidate tools first is common [7, 28, 30], and tool retrieval has been benchmarked as its own task [36]. We use BM25 [32], a lexical baseline that remains competitive out of domain [37], and we use its score a second time, as the refusal gate.

Out-of-scope detection and abstention. Intent classification has standard datasets [5], one of them with an out-of-scope class [24], and methods that learn a decision boundary per known class [42]. More broadly, selective prediction answers only above a confidence threshold [13, 23], the maximum softmax probability is a baseline signal for it [19], and abstention in language models has been surveyed by Wen et al. [39]. Our gate is a selective classifier whose confidence is a retrieval score, not a model probability.

Choosing by option probability. Multiple-choice answers can be read from the probabilities a model assigns to each option [4], to label words [34], or to option symbols in a single forward pass [33]. These probabilities carry label and position biases [29, 44, 45], compete across surface forms [20], and need calibration to mean what they say [18, 22]. Our chooser generates its call greedily instead; the probability of the chosen name is one of the signals we record but have not measured.

Structured output. Constrained decoding guarantees a well-formed output [15, 40], and its implementations differ in coverage and fidelity [16]. A well-formed answer can still be wrong. We check form after generation, by comparison against the catalog, and say explicitly that the check establishes form and not correctness.

Keeping an agent inside its boundary. Prompt injection turns untrusted content into instructions [17], and agent benchmarks measure how often defences hold [8]. Defences that hold by construction separate the model that reads untrusted data from the one that plans, and enforce capabilities in code [2, 9]. Our design is in that family: the chooser reads only the person’s own message, and its output space is closed and checked.

Fine-tuning variance and written data. Fine-tuning outcomes vary substantially with the random seed [10, 27], and benchmark comparisons that ignore that variance can rank noise [3]. We train with LoRA [21] on messages written by other models, in the manner of self-generated instruction data [38], and use a model as a judge only where no label exists [46].

3 Design

3.1 The catalog

The catalog is a list of use cases, each with a name, a kind (command or query), an engineering *intent*, the parameters it *accepts* with their types or enumerated values, and a short description *in words*, one sentence in English and one in Portuguese, in the words a person would use. The catalog studied here has thirteen use cases for managing a fictional social account: refreshing an inbox and feed, staging a direct message, a post, a reply to a comment, a profile change, and reading posts, drafts, conversations and similar. Every command *stages*: it prepares an action that a person then accepts, edits or rejects. Nothing acts on its own.

Tools are derived: one per use case, its name, its description, its parameters as a JSON schema. Nothing is written by hand per tool.

3.2 One turn

A message travels four stages, and every stage writes its number into a record whether or not the next stage runs.

1. **Narrowing.** Each use case is a document made of its description, its intent, and the training messages labelled with it. The message is scored against every document with BM25 over lowercased, accent-stripped words, and the four highest are kept.
2. **The gate.** If the top score is below a threshold, the turn is answered with none and no model is called. The threshold is fitted once on the training set.
3. **The chooser.** The model is offered the four candidates as tools and decodes greedily. Its output is parsed into a name and arguments, or no call.
4. **The checks.** The name must parse, exist in the catalog, and be among the four offered; otherwise the turn is refused. An argument must be a declared parameter, an enumerated value must be declared, and a text value must appear in the message; otherwise that argument alone is dropped and left for the person to fill.

The checks are comparisons. They say the answer is well formed and inside the catalog; they do not say it is right. Correctness is settled by the person, who sees a staged command and accepts, edits or rejects it. That is what makes an accuracy below 100% tolerable, and it is why every command in the catalog stages.

4 Experimental setup

Model and training. `unsloth/functiongemma-270m-it` at revision `79eaaa8`, an ungated copy of a 270M function-calling model from the Gemma 3 family [14]. Greedy decoding, MLX on an Apple M3 Pro laptop. LoRA [21] through `mlx_lm.lora` with the prompt masked, batch 4, learning rate 10^{-4} , two passes over the training rows, and seed 0 unless stated. A training run takes about fifteen minutes.

Training data. Positives and near misses were written by `openai/gpt-oss-120b` from the catalog, at temperature 1.2 under a strict schema. Near misses, messages close to a use case that it does not serve, were labelled by a second model, `qwen/qwen3.8-27b`, which failed to produce valid JSON in roughly one call in five; failed batches were dropped and never retried. A text argument the message does not state is removed from written data rather than discarding the message. The final training set has 1,245 rows.

Tests. Three tests, none seen by any training run before it was measured. *Old*: 28 messages (16 fitting one use case, 12 fitting none), held out from the catalog’s 55 original examples in English and Portuguese. *Test-1*: 51 messages (39 and 12), written by `qwen/qwen3.8-27b` and `llama3.1:8b`, and labelled blind by `gpt-oss-120b`, which agreed on 49. *Test-2*: 56 messages (39 and 17), written by `llama3.1:8b`, a third model family, with messages the two labellers disagreed on removed. No test was written by a person.

Declared signals. Every experiment was declared in writing, with a hypothesis and a yes-or-no signal, before its code was written, and no threshold moved after a run. The bar is ours, argued from what a person would tolerate from an assistant: at least 80% of messages that fit a use case answered with the right one, and at least 90% of messages that fit none answered with none.

Cost. The whole study made 155 hosted model calls, 12 of which failed, for US\$0.2198. No training ran anywhere but the laptop.

#	What changed	Test	Fits one	Fits none	Signal
1	LoRA on 39 examples, no “none” taught	old	6/16	3/12	no
2	941 written messages, judged near misses	old	11/16	6/12	no
3	BM25 narrowing to four, descriptions in words	test-1	8/39	11/12	no
4	artificial “none” rows removed	test-1	32/39	9/12	no
5	139 more near misses	test-2	30/39	4/17	no
6	BM25 threshold refuses before the model	test-2	34/39	15/17	no, by one
7	four use cases held out of training	test-2	2/12 on the four	15/17 gated	no
8	seed only, three times over #5’s data	test-2	30, 28, 33	4, 13, 10	no

Table 1: The chooser experiments. Untuned and offered all thirteen use cases, the model answered 2 of 16 messages that fit one and abstained from nearly everything, including what it should have answered; offered four, 4 of 16.

5 Results

Table 1 lists the eight chooser experiments that were run. A ninth, measuring the untuned model among four candidates, was answered inside the first and superseded. None of the eight cleared its signal; the closest missed by one message.

5.1 The seed governs everything else

Experiment 8 trained one configuration three times, changing nothing but the seed. On the 39 messages that fit a use case, the three runs answer 30, 28 and 33 correctly, a spread of 13 points. On the 17 that fit none, without the gate, they refuse 4, 13 and 10, a spread of 52 points. This is the variance reported for fine-tuning generally [10, 27], at a scale where it swamps every other change we made. We therefore withdraw every ranking of adapters in Table 1 whose difference is smaller than these spreads: the rows stand as what each run measured, but no sentence comparing them does. In particular, the drop in refusal from #4 to #5, which looked like an effect of adding near misses, was seed 0 of a configuration whose seed 1 refuses 76%.

What survives is that choosing among four works at this size: from 2 of 16 untuned to between 72% and 90% on a test written by a third model family and never seen.

5.2 Refusal is a retrieval question

No training set taught the model to refuse reliably. The attempts gave 75%, 59% and 24%, and three seeds of the last gave 24%, 76% and 59%: the data was not controlling that number.

A threshold on the BM25 score that narrowing already computes did. Fitted once, on the training set alone, at 8.891 over 1,081 positives and 298 near misses, and never tuned on a test, it takes the same three seeds from 24%, 76% and 59% refusal to 88%, 94% and 88%, and collapses the spread from 52 points to 6. Fifteen of the seventeen off-catalog messages never reach the model. Whether a message is inside or outside the catalog behaves like a retrieval question; which use case it asks for is where the model sits.

The refusals that still fail are ordinary: “make me a coffee”, “pick up some milk”, answered with the direct-message use case. Two of the three messages the gate gets wrong are arguably mislabelled, and with seventeen messages deciding a bar, one label is six points.

Adapter	Agree	Disagree	Gate said none
#4, all use cases trained	32/38 (84%)	2/2	15/16 (94%)
#5, more near misses	29/34 (85%)	0/6	15/16 (94%)
#7, four held out	21/25 (84%)	1/15	15/16 (94%)

Table 2: Accuracy by agreement between the retriever’s first place and the chooser on test-2.

5.3 The retrieval floor

Choosing by retrieval alone, with no model and nothing trained, names the right use case for 33 of the 39 messages (85%) when the answer is narrowing’s first place, and for 28 (72%) when the five nearest training messages vote. The three tuned seeds give 30, 28 and 33. On this test and this catalog the tuned model is worth about one message in thirty-nine over the retriever that was meant only to shortlist for it, and two of the three seeds fall below the retriever.

This is not a verdict on the model, for two reasons. The model binds arguments and the retriever cannot; argument binding, the part of the job the design actually needs, peaked at 7 of 16 exact bindings early in the study and was not taken up again. And the retriever’s documents are made of messages one model wrote from this catalog, tested on messages another model wrote; a person’s own phrasing may separate the two.

5.4 A use case is a build step

A newly declared use case is a label the model never saw in training, the condition of zero-shot classification [41]. Experiment 7 trained without four of the thirteen use cases while narrowing still found them and they were still offered as tools. The model chose them for 2 of the 12 messages asking for them, against 8 and 11 for adapters that had trained on them, while holding 78% on the nine it had seen. Nine of the ten misses called a trained neighbour; one said none.

Retrieval depends on training data too. Treating each use case in turn as newly declared, with its written messages removed from the narrowing documents and only its two-sentence description left, the right use case is kept among the four for 15 of 39 messages, against 38 of 39 with its written messages. Five of the thirteen are never found. The gate is unaffected: the message still passes, and the model is offered four candidates that do not include the right one.

Both halves say the same thing. The catalog derives the tools, the prompts and the narrowing documents, but the chooser is compiled from the catalog rather than reading it: about half a cent of hosted writing and fifteen minutes of training per change. Small, but a build step, and the design should say so rather than imply that a use case can be declared at runtime.

5.5 What the first pass already knows

Every message crosses two opinions before anything is shown: the retriever’s first place and the chooser’s. Table 2 splits three recorded runs by whether they agree.

The agree bucket is 84% to 85% under all three adapters and the gate is 94% every time; only the disagree column moves. When a use case is new to the chooser, overriding the retriever is almost always a neighbour: on the held-out run, disagreement marks 15 of 56 turns and contains 14 of the 19 errors. When everything was trained, the chooser overrides rarely and is right when it does. The flag is worth little while the catalog is stable and worth most exactly when a use case is new, which is the condition nothing else here detects. The samples are 2, 6 and 15 messages.

Use cases	Recall at four	Right one first	Refused	Mean rank
4	100.0%	91.7%	85.3%	1.12
6	100.0%	94.4%	82.4%	1.08
8	98.3%	88.9%	81.4%	1.19
10	97.5%	87.8%	85.8%	1.23
12	97.2%	84.5%	82.8%	1.33
13	97.4%	84.6%	88.2%	1.33
Slope per use case	−0.35	−1.04	−0.11	

Table 3: Retrieval as the catalog grows (selected sizes; the full table has every size from 4 to 13). Thirteen is the whole catalog and reproduces three numbers reported separately above.

A second signal reads from a single turn. Replaying the three runs through the turn pipeline, with the recorded output standing in for the model, reproduces every candidate set and chosen name, and adds the arguments the checks drop. An argument naming a parameter the use case does not declare appears four times, on three turns, only in the run trained without four use cases, and all three turns ask for one of the held-out four. It never appears in the 112 turns of the other two runs. Three turns is not a rate.

Neither signal needs a second model. We did not use one to review choices: every one of the ten held-out misses is well formed, in the catalog, among the offered tools and wrong, and a reviewer of form would pass all ten.

5.6 How the shortlist scales

The chooser always sees four candidates, so its prompt, latency and accuracy are independent of the catalog’s size. Narrowing and the gate are not: every use case added is another document competing for the same words. We subset the catalog from four to thirteen use cases, twelve random subsets per size, rebuilding narrowing and refitting the threshold for each, and evaluate on the test-2 messages asking for a use case in the subset and on all seventeen asking for none.

The declared signal, both recall at four and refusal falling by less than one point per use case added, was met: −0.35 and −0.11. The shortlist is the part of the design that scales, as far as subsetting can show. The retriever’s first place, which the signal did not judge, falls three times faster, about a point per use case: the right use case stays in the shortlist but is less often first. The quantity the model is paid to fix, picking among four when the first is not obviously right, is the quantity that degrades with size, and the comparison in §5.3 was made at the size least favourable to the model. That the model’s value grows with the catalog is argued, not measured: no model was trained on a larger catalog. And subsetting goes down while the question goes up; six use cases drawn from thirteen are not the six nearest neighbours a real catalog of sixty would contain, so the slopes are a lower bound.

6 Discussion

The measurements settle a division of labour that the design had only argued. Whether a message is inside the catalog is answered by retrieval and a threshold, and that answer is the only reproducible number in the pipeline. Which use case the message asks for is where the model sits, with a spread wide enough to straddle the retriever that shortlists for it. What keeps the answer inside the catalog is a comparison, and the comparison is the one part that needs no measurement beyond its tests:

the untuned model named three tools it had never been offered, and the checks caught all three.

This has consequences for how such a system is monitored. A blended accuracy is mostly made of the agree bucket and the gate, which do not move, and hides the disagree bucket, which does. A chooser-only accuracy credits the model with the pipeline. The label that matters, whether the person accepted the staged command, arrives for free because every command stages; it is also biased upward, since a person accepting a plausible neighbour labels a wrong choice right, and nine of the ten held-out misses were plausible neighbours.

Two directions follow from the related work and were not tried. Reading option probabilities in a single pass [33] would give the chooser a confidence to threshold, subject to the biases and calibration issues documented for that mechanism [18, 44, 45]. And an untrained model of a few billion parameters, read this way, might choose a newly declared use case far better than 2 of 12, which would make a use case declarative again. Both are hypotheses.

7 Limitations

- **One catalog of thirteen use cases**, fictional, written by the author, all of whose commands stage. A catalog with an irreversible command loses both the safety and the free label.
- **No person wrote a test**. Three models wrote the messages and two labelled them; in test-2, messages the labellers disagreed on were removed, which removes the hardest cases.
- **Small samples**. Seventeen off-catalog messages decide the refusal bar, so one message is six points. Three seeds is the smallest sample that can show a spread and the worst for estimating it.
- **Arguments were barely measured**. Every headline number is about naming the right use case.
- **The gate’s threshold belongs to its training set**, whose messages are also narrowing’s documents; the threshold moves whenever that set grows. Its fitting objective weighs both halves equally while the bar does not, and refitting it now, after seeing the result, would be fitting the test.
- **No adversarial message was run**. None of the 119 test messages attempts to redirect the agent. By construction the chooser reads only the person’s own message, never a stranger’s, but the rate at which a crafted message stages a wrong and plausible command is unmeasured. A model inside a use case, which would read untrusted content, does not exist yet and nothing here speaks to it.
- **A single judge model** labelled the near misses and failed on one call in five. No person read a sample of the positives or the near misses.

8 Conclusion

A 270M model on a laptop can choose among four declared use cases well enough to be useful when every command it stages is reviewed by the person, but on this catalog it barely beats the retrieval step that shortlists for it, it cannot be trained to refuse, and its accuracy moves 13 points with the seed. Refusal belongs to a threshold on retrieval; form belongs to comparisons against the catalog; correctness belongs to the person. What the model adds is the choice among close neighbours,

which is the part that should grow with the catalog, and the one this study measured at its least favourable size.

Reproducibility. Every experiment’s declaration, results and marked doubts, the scripts, the data, and one record per model call with its model id, input, output and date are kept in the accompanying repository. Nothing retried: a failed call was reported and the run stopped there.

References

- [1] Berkeley Function Calling Leaderboard. Berkeley function calling leaderboard. <https://gorilla.cs.berkeley.edu/leaderboard.html>, 2024. Cited as the benchmark the chooser’s model card reports; not on arXiv.
- [2] Luca Beurer-Kellner, Beat Buesser, Ana-Maria Crețu, et al. Design patterns for securing LLM agents against prompt injections, 2025.
- [3] Xavier Bouthillier, Pierre Delaunay, Mirko Bronzi, et al. Accounting for variance in machine learning benchmarks. In *Proceedings of MLSys*, 2021.
- [4] Tom B. Brown et al. Language models are few-shot learners, 2020.
- [5] Iñigo Casanueva, Tadas Temčinas, Daniela Gerz, Matthew Henderson, and Ivan Vulić. Efficient intent detection with dual sentence encoders, 2020.
- [6] Wei Chen and Zhiyuan Li. Octopus v2: On-device language model for super agent, 2024.
- [7] Yanfei Chen, Jinsung Yoon, Devendra Singh Sachan, et al. Re-Invoke: Tool invocation rewriting for zero-shot tool retrieval, 2024.
- [8] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, et al. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *NeurIPS Datasets and Benchmarks*, 2024.
- [9] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, et al. Defeating prompt injections by design, 2025.
- [10] Jesse Dodge, Gabriel Ilharco, Roy Schwartz, Ali Farhadi, Hannaneh Hajishirzi, and Noah Smith. Fine-tuning pretrained language models: Weight initializations, data orders, and early stopping, 2020.
- [11] Lutfi Eren Erdogan, Nicholas Lee, Siddharth Jha, et al. TinyAgent: Function calling at the edge, 2024.
- [12] Tiantian Gan and Qiyao Sun. RAG-MCP: Mitigating prompt bloat in LLM tool selection via retrieval-augmented generation, 2025.
- [13] Yonatan Geifman and Ran El-Yaniv. Selective classification for deep neural networks. In *Advances in NeurIPS*, 2017.
- [14] Gemma Team. Gemma 3 technical report, 2025.
- [15] Saibo Geng, Martin Josifoski, Maxime Peyrard, and Robert West. Grammar-constrained decoding for structured NLP tasks without finetuning. In *Proceedings of EMNLP*, 2023.

- [16] Saibo Geng, Hudson Cooper, Michal Moskal, et al. JSONSchemaBench: A rigorous benchmark of structured outputs for language models, 2025.
- [17] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection, 2023.
- [18] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of ICML*, 2017.
- [19] Dan Hendrycks and Kevin Gimpel. A baseline for detecting misclassified and out-of-distribution examples in neural networks. In *Proceedings of ICLR*, 2017.
- [20] Ari Holtzman, Peter West, Vered Shwartz, Yejin Choi, and Luke Zettlemoyer. Surface form competition: Why the highest probability answer isn’t always right. In *Proceedings of EMNLP*, 2021.
- [21] Edward J. Hu, Yelong Shen, Phillip Wallis, et al. LoRA: Low-rank adaptation of large language models. In *Proceedings of ICLR*, 2022.
- [22] Saurav Kadavath et al. Language models (mostly) know what they know, 2022.
- [23] Amita Kamath, Robin Jia, and Percy Liang. Selective question answering under domain shift. In *Proceedings of ACL*, 2020.
- [24] Stefan Larson, Anish Mahendran, Joseph J. Peper, Christopher Clarke, Andrew Lee, Parker Hill, Jonathan K. Kummerfeld, Kevin Leach, Michael A. Laurenzano, Lingjia Tang, and Jason Mars. An evaluation dataset for intent classification and out-of-scope prediction. In *Proceedings of EMNLP-IJCNLP*, 2019.
- [25] Minghao Li, Yingxiu Zhao, Bowen Yu, et al. API-Bank: A comprehensive benchmark for tool-augmented LLMs, 2023.
- [26] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts, 2023.
- [27] Marius Mosbach, Maksym Andriushchenko, and Dietrich Klakow. On the stability of fine-tuning BERT: Misconceptions, explanations, and strong baselines. In *Proceedings of ICLR*, 2021.
- [28] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive APIs, 2023.
- [29] Pouya Pezeshkpour and Estevam Hruschka. Large language models sensitivity to the order of options in multiple-choice questions, 2023.
- [30] Yujia Qin, Shihao Liang, Yining Ye, et al. ToolLLM: Facilitating large language models to master 16000+ real-world APIs, 2023.
- [31] Changle Qu, Sunhao Dai, Xiaochi Wei, et al. Tool learning with large language models: A survey, 2024.

- [32] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009. doi: 10.1561/15000000019.
- [33] Joshua Robinson, Christopher Michael Rytting, and David Wingate. Leveraging large language models for multiple choice question answering. In *Proceedings of ICLR*, 2023.
- [34] Timo Schick and Hinrich Schütze. Exploiting cloze questions for few shot text classification and natural language inference. In *Proceedings of EACL*, 2021.
- [35] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools, 2023.
- [36] Zhengliang Shi, Yuhan Wang, Lingyong Yan, et al. Retrieval models aren’t tool-savvy: Benchmarking tool retrieval for large language models, 2025.
- [37] Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. BEIR: A heterogenous benchmark for zero-shot evaluation of information retrieval models. In *NeurIPS Datasets and Benchmarks*, 2021.
- [38] Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, et al. Self-Instruct: Aligning language models with self-generated instructions. In *Proceedings of ACL*, 2023.
- [39] Bingbing Wen, Jihan Yao, Shangbin Feng, et al. Know your limits: A survey of abstention in large language models, 2024.
- [40] Brandon T. Willard and Rémi Louf. Efficient guided generation for large language models, 2023.
- [41] Wenpeng Yin, Jamaal Hay, and Dan Roth. Benchmarking zero-shot text classification: Datasets, evaluation and entailment approach. In *Proceedings of EMNLP-IJCNLP*, 2019.
- [42] Hanlei Zhang, Hua Xu, and Ting-En Lin. Deep open intent classification with adaptive decision boundary. In *Proceedings of AAAI*, 2021.
- [43] Jianguo Zhang, Kazuma Hashimoto, Yao Wan, Zhiwei Liu, Ye Liu, Caiming Xiong, and Philip S. Yu. Are pretrained transformers robust in intent classification? A missing ingredient in evaluation of out-of-scope intent detection, 2021.
- [44] Tony Z. Zhao, Eric Wallace, Shi Feng, Dan Klein, and Sameer Singh. Calibrate before use: Improving few-shot performance of language models. In *Proceedings of ICML*, 2021.
- [45] Chujie Zheng, Hao Zhou, Fandong Meng, Jie Zhou, and Minlie Huang. Large language models are not robust multiple choice selectors. In *Proceedings of ICLR*, 2024.
- [46] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *NeurIPS Datasets and Benchmarks*, 2023.